Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 2

По дисциплине: «КиСБ»

Тема: «Избыточное кодирование данных в информационных системах»

Выполнила:
Студентка 4 курса
Группы АС-65
Рапин Е. Ю.
Проверила:
Хацкевич А. С.

Брест 2026

Цель: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.

Ход работы

Часть 1:

1. Закрепить теоретические знания по использованию методов помехоустойчивого кодирования для повышения надежности передачи и хранения в памяти компьютера двоичных данных.
2. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 или 4.
3. Результаты выполнения лабораторной работы оформить в виде описания разработаного приложения, методики выполнения экспериментов с использованием приложения и результатов эксперимента.

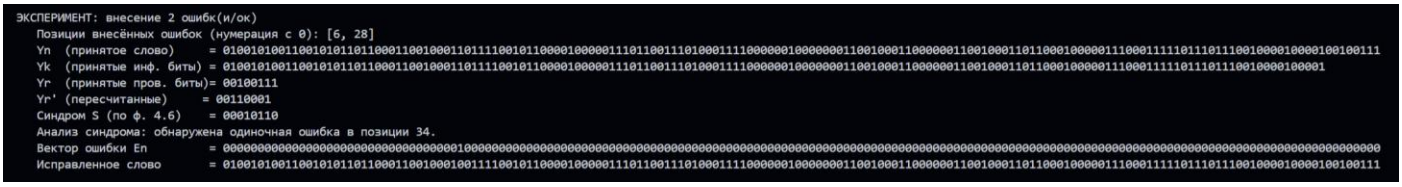
```
Источник данных: файл 'text.txt'

1. Исходное информационное сообщение (двоичное):
   Xk = 0100100001100101011011000110110001101111000110111001101000111100000010000000110010001100000011001000110110001000001110001111101110110010000100001
   k  = 160  (длина информационного слова)
```

Сообщение из обрабатываемого файла: «Hello, мир 2026 год!» (20 символов*8 бит = 160 бит).

```
2. Параметры кода:
   r      = 8   (число проверочных / избыточных символов)
   n      = 168   (длина кодового слова, n = k + r)
   d_min  = 3   (минимальное кодовое расстояние)
   t_o    = 1   (обнаруживающая способность)
   t_u    = 1   (исправляющая способность)
   Степень кодирования (code rate) R = k/n = 160/168 ≈ 0.9524
```

```
3. Проверочная матрица Хемминга H размера 8×168:
   H = [P' | I]   (формула 4.4)
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```



Программа реализует полный цикл кодирования/декодирования кодом Хемминга с d_min = 3 и корректно работает при любом k. Для одиночной ошибки (позиция 163) синдром однозначно указывает позицию, ошибка исправляется. Для двух ошибок (позиции 6 и 28) синдром совпал со столбцом №34 — произошло ложное «исправление», которое было обнаружено сравнением с исходным Xn. Это иллюстрирует фундаментальное ограничение кода Хемминга с d_min = 3: он гарантированно исправляет только одиночные ошибки, а двойные может либо обнаружить, либо замаскировать под одиночную. В реальной системе, где Xn приёмнику неизвестно, ложное исправление осталось бы незамеченным — для надёжной работы с двойными ошибками требуется код с d_min = 4 (расширенный код Хемминга) либо другой помехоустойчивый код.

Код программы:

```
import random
from typing import List, Tuple, Optional

def fact(n: int) -> int:
    """Факториал n."""
    if n <= 1:
        return 1
    return fact(n - 1) * n

def binom(wt: int, r: int) -> int:
    """Бином Ньютона C(r, wt)."""
    if wt < 0 or wt > r:
        return 0
    return fact(r) // (fact(wt) * fact(r - wt))

def int_to_bin_str(num: int, width: int = 0) -> str:
    """Двоичное представление числа."""
    s = bin(num)[2:]
    return s.zfill(width) if width > 0 else s

def weight(bin_str: str) -> int:
    """Количество единиц в строке."""
    return bin_str.count('1')

def bits_to_str(bits: List[int]) -> str:
    return ''.join(str(b) for b in bits)

# Построение проверочной матрицы H (d_min = 3)

def find_r(k: int) -> int:
    """Число проверочных символов r из 2^r >= k + r + 1."""
    r = 1
    while (1 << r) < (k + r + 1):
        r += 1
    return r

def build_P_matrix(r: int, k: int) -> List[List[int]]:
    """Подматрица P' размера r × k. Столбцы начиная с веса 2."""
    P = [[0] * k for _ in range(r)]
    count = 0
    wt = 2

    while count < k and wt <= r:
        for num in range(1, 1 << r):
            if count >= k:
                break
            bin_repr = int_to_bin_str(num, width=r)
            if weight(bin_repr) == wt:
                for i in range(r):
```

```
                    P[i][count] = int(bin_repr[i])
                count += 1
        wt += 1

    if count < k:
        raise ValueError(f"Не удалось построить {k} столбцов для r={r}.")
    return P

def build_H_matrix(k: int, r: Optional[int] = None
                   ) -> Tuple[List[List[int]], int, int]:
    """Проверочная матрица H = [P' | I], n = k + r."""
    if r is None:
        r = find_r(k)
    n = k + r

    P = build_P_matrix(r, k)
    H = [[0] * n for _ in range(r)]

    for i in range(r):
        for j in range(k):
            H[i][j] = P[i][j]
        for j in range(r):
            H[i][k + j] = 1 if i == j else 0

    return H, n, r

def matrix_to_str(M: List[List[int]]) -> str:
    return '\n'.join(' '.join(str(x) for x in row) for row in M)

# Кодирование

def compute_parity_bits(Xk: List[int], H: List[List[int]],
                        k: int, r: int) -> List[int]:
    """Вычисление проверочных битов Xr (сумма по модулю 2)."""
    Xr = [0] * r
    for i in range(r):
        s = 0
        for j in range(k):
            s ^= (Xk[j] & H[i][j])
        Xr[i] = s
    return Xr

def encode(Xk: List[int], H: List[List[int]],
           k: int, r: int) -> List[int]:
    """Кодовое слово Xn = [Xk | Xr]."""
    return Xk + compute_parity_bits(Xk, H, k, r)

# Синдром и исправление ошибок

def compute_syndrome(Yn: List[int], H: List[List[int]],
                     r: int, n: int) -> List[int]:
    """Синдром S через пересчёт проверочных битов."""
    k = n - r
    Yr_recomputed = compute_parity_bits(Yn[:k], H, k, r)
    Yr_received = Yn[k:]
    return [1 if Yr_received[i] != Yr_recomputed[i] else 0 for i in range(r)]

def syndrome_matrix_multiply(Yn: List[int], H: List[List[int]],
                             r: int, n: int) -> List[int]:
    """Синдром S = H · Yn^T (mod 2)."""
    S = [0] * r
    for i in range(r):
        s = 0
        for j in range(n):
            s ^= (H[i][j] & Yn[j])
        S[i] = s
    return S

def syndrome_to_error_position(S: List[int], H: List[List[int]],
                               n: int) -> int:
    """
    Позиция ошибки по синдрому:
    -1 — ошибок нет,
    >=0 — позиция одиночной ошибки,
    -2 — вероятно две ошибки.
```

```
    """
    if all(s == 0 for s in S):
        return -1
    for j in range(n):
        if [H[i][j] for i in range(len(S))] == S:
            return j
    return -2

def build_error_vector(position: int, n: int) -> List[int]:
    """Унарный вектор ошибки En."""
    En = [0] * n
    if 0 <= position < n:
        En[position] = 1
    return En

def correct_error(Yn: List[int], En: List[int]) -> List[int]:
    """Исправление: Yn XOR En."""
    return [y ^ e for y, e in zip(Yn, En)]

# Внесение ошибок

def introduce_errors(Xn: List[int], num_errors: int
                     ) -> Tuple[List[int], List[int]]:
    """Внесение num_errors случайных ошибок."""
    n = len(Xn)
    Yn = Xn[:]
    positions = random.sample(range(n), num_errors) if num_errors > 0 else []
    for pos in positions:
        Yn[pos] ^= 1
    return Yn, sorted(positions)

# Формирование информационного слова

def text_to_bits(text: str) -> List[int]:
    """Текст → биты (кодировка cp1251)."""
    bits: List[int] = []
    for ch in text:
        try:
            b = ch.encode('cp1251')[0]
        except UnicodeEncodeError:
            b = ord(ch) & 0xFF
        for i in range(7, -1, -1):
            bits.append((b >> i) & 1)
    return bits

def bits_from_text(text: str, min_bits: int = 16) -> List[int]:
    """Информационное слово длиной не менее min_bits."""
    bits = text_to_bits(text)
    while len(bits) < min_bits:
        bits.append(0)
    return bits

def bits_from_file(filename: str, min_bits: int = 16) -> List[int]:
    """Чтение файла и преобразование в биты."""
    with open(filename, 'r', encoding='utf-8') as f:
        text = f.read().strip()
    if not text:
        raise ValueError(f"Файл '{filename}' пуст.")
    return bits_from_text(text, min_bits)

# Основной эксперимент

def run_experiment(info_bits: List[int],
                   num_errors_list: Optional[List[int]] = None,
                   seed: Optional[int] = 42) -> None:
    """Кодирование, внесение 0/1/2 ошибок, декодирование."""
    if seed is not None:
        random.seed(seed)
    if num_errors_list is None:
        num_errors_list = [0, 1, 2]

    def out(s: str = '') -> None:
        print(s)

    k = len(info_bits)
```

```
H, n, r = build_H_matrix(k)
Xn = encode(info_bits, H, k, r)
Xr = Xn[k:]

d_min = 3
t_o = (d_min - 1) // 2
t_u = (d_min - 1) // 2

out()
out('1. Исходное информационное сообщение (двоичное):')
out(f'   Xk = {bits_to_str(info_bits)}')
out(f'   k  = {k}')

out()
out('2. Параметры кода:')
out(f'   r     = {r}')
out(f'   n     = {n}')
out(f'   d_min = {d_min}')
out(f'   t_o   = {t_o}')
out(f'   t_u   = {t_u}')
out(f'   R = k/n = {k}/{n} ≈ {k/n:.4f}')

out()
out(f'3. Проверочная матрица Хемминга H размера {r}×{n}:')
out(matrix_to_str(H))

out()
out('4. Избыточные символы и кодовое слово:')
out(f'   Xr = {bits_to_str(Xr)}')
out(f'   Xn = {bits_to_str(Xn)}')

for num_err in num_errors_list:
    out()
    out(f'ЭКСПЕРИМЕНТ: внесение {num_err} ошибк(и/ок)')

    Yn, err_pos = introduce_errors(Xn, num_err)
    Yk = Yn[:k]
    Yr_received = Yn[k:]

    out(f'   Позиции ошибок (с 0): {err_pos if err_pos else "нет"}')
    out(f'   Yn  = {bits_to_str(Yn)}')
    out(f'   Yk  = {bits_to_str(Yk)}')
    out(f'   Yr  = {bits_to_str(Yr_received)}')

    Yr_recomputed = compute_parity_bits(Yk, H, k, r)
    out(f"   Yr' = {bits_to_str(Yr_recomputed)}")

    S = compute_syndrome(Yn, H, r, n)
    S_mat = syndrome_matrix_multiply(Yn, H, r, n)
    out(f'   Синдром S = {bits_to_str(S)}')
    if S != S_mat:
        out(f'   !!! Расхождение с матричным умножением: {bits_to_str(S_mat)}')

    pos = syndrome_to_error_position(S, H, n)

    if pos == -1:
        out('   S = 0 → ошибок не обнаружено.')
        En = [0] * n
        Xn_corr = Yn[:]
        out(f'   En = {bits_to_str(En)}')
        out(f'   Исправленное слово = {bits_to_str(Xn_corr)}')
        out('   Информационное слово восстановлено верно.')

    elif pos >= 0:
        out(f'   Обнаружена одиночная ошибка в позиции {pos}.')
        En = build_error_vector(pos, n)
        Xn_corr = correct_error(Yn, En)
        out(f'   En = {bits_to_str(En)}')
        out(f'   Исправленное слово = {bits_to_str(Xn_corr)}')
        if Xn_corr == Xn:
            out('   >>> Ошибка успешно исправлена.')
        else:
            out('   >>> ВНИМАНИЕ: слово не совпало с исходным (возможна кратная ошибка).')

    else:
```

```
            out('   Синдром не совпал ни с одним столбцом H.')
            out('   Вероятно, две (или более) ошибки.')
            out('   Код с d_min = 3 обнаруживает, но не исправляет двойную ошибку.')
            En = [0] * n
            out(f'   En = {bits_to_str(En)} (исправление не производилось)')

    out()

def main():
    try:
        info_bits = bits_from_file(
            r'D:\University\PythonProjects\KISBLab2_1\text.txt', min_bits=16
        )
        print("Источник данных: файл 'text.txt'")
    except FileNotFoundError:
        text = "Hello, мир 2026 год!"
        info_bits = bits_from_text(text, min_bits=16)
        print(f"Источник данных: строка в коде ({text!r})")

    run_experiment(
        info_bits,
        num_errors_list=[0, 1, 2],
        seed=42,
    )

if __name__ == '__main__':
    main()
```

Часть 2:

1. Закрепить теоретические знания по использованию итеративных кодов для повышения надежности передачи и хранения в памяти компьютера двоичных данных.
2. Разработать приложение для кодирования/декодирования двоичной информации итеративным кодом с различной относительной избыточностью кодовых слов.
3. Результаты выполнения лабораторной работы оформить виде описания разработанного приложения, методики выполнения экспериментов с использованием приложения и результатов эксперимента.

Практическое задание

Разработать собственное приложение, которое позволяет выполнять следующие операции:

1. вписывать произвольное двоичное представление информационного слова *Xk* (кодируемой информации) длиной *k* битов в двумерную матрицу размерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2);
2. вычислять проверочные биты (биты паритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов);
3. формировать кодовое слово *Xn* присоединением избыточных символов к информационному слову;
4. генерировать ошибку произвольной кратности (*i*, *i* > 0), распределенную случайным образом среди символов слова *Xn*, в результате чего формируется кодовое слово *Yn*;
5. определять местоположение ошибочных символов итеративным кодом в слове *Yn* в соответствии с используемыми группами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления – слово *Yn'*);
6. выполнять анализ корректирующей способности используемого кода (количественная оценка) путем сравнения соответствующих слов *Xn* и *Yn'*; результат анализа может быть

представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ошибки этой кратности, которые правильно обнаружены и которые правильно скорректированы.

Вариант	Длина информационного слова (бит), k	k_1	k_2	z	Количество групп паритетов
1	16	4	4	–	2; 3
		8	2	-	2; 3
		4	2	2	2; 3; 4; 5
		2	4	2	2; 3; 4; 5

1. Формирование кодового слова

```
Выберите конфигурацию [1]: 1

Выбрано: k1=4, k2=4, k=16

Количество групп паритетов:
  2 — строки + столбцы
  3 — строки + столбцы + общий
  4 — строки + столбцы + главные диагонали + побочные диагонали
  5 — всё выше + общий паритет
Введите число групп (2–5) [3]: 3
Используется 3 групп паритетов

Введите информационное слово Xk (16 бит, только 0 и 1)
или нажмите Enter для случайного слова:

Сгенерировано: 0010101010011001

Информационная матрица Xk:
0 0 1 0
1 0 1 0
1 0 0 1
1 0 0 1

Кодовое слово Xn (длина n=25):
0010101010011001100010000
```

2. Исправление одиночной ошибки

```
Введите кратность ошибки i (целое >0) [1]: 1

Внесены ошибки в позиции: [1]
Слово с ошибками Yn:
0110101010011001100010000

Синдромы:
  Строк:         [1 0 0 0]
  Столбцов:      [0 1 0 0]
  Общего:        0

Исправленные позиции: [np.int64(1)]
Исправленное слово Yn*:
0010101010011001100010000

>>> Ошибки успешно исправлены. Yn* совпадает с Xn.
```

3. Обнаружение (без исправления) двойной ошибки

```
Выберите конфигурацию [1]: 1

Выбрано: k1=4, k2=4, k=16

Количество групп паритетов:
  2 — строки + столбцы
  3 — строки + столбцы + общий
  4 — строки + столбцы + главные диагонали + побочные диагонали
  5 — всё выше + общий паритет
Введите число групп (2–5) [3]: 3
Используется 3 групп паритетов

Введите информационное слово Xk (16 бит, только 0 и 1)
или нажмите Enter для случайного слова:

Сгенерировано: 0011010000111010

Информационная матрица Xk:
0 0 1 1
0 1 0 0
0 0 1 1
1 0 1 0

Кодовое слово Xn (длина n=25):
0011010000111010010011100

Паритеты:
  Строковые:     [0 1 0 0]
  Столбцовые:    [1 1 1 0]
  Общий:         0

Введите кратность ошибки i (целое >0) [1]: 2

Внесены ошибки в позиции: [16, 19]
Слово с ошибками Yn:
0011010000111010110111100
```

```
Синдромы:
  Строк:         [1 0 0 1]
  Столбцов:      [0 0 0 0]
  Общего:        0

Исправленные позиции: []
Исправленное слово Yn*:
0011010000111010110111100

>>> Полное исправление не удалось (характерно для кратности > 1).
```

4. Работа с другой размерностью

```
Выберите конфигурацию [1]: 2

Выбрано: k1=8, k2=2, k=16

Количество групп паритетов:
  2 — строки + столбцы
  3 — строки + столбцы + общий
  4 — строки + столбцы + главные диагонали + побочные диагонали
  5 — всё выше + общий паритет
Введите число групп (2–5) [3]: 3
Используется 3 групп паритетов

Введите информационное слово Xk (16 бит, только 0 и 1)
или нажмите Enter для случайного слова:

Сгенерировано: 1010011001110101

Информационная матрица Xk:
1 0
1 0
0 1
1 0
0 1
1 1
0 1
0 1

Кодовое слово Xn (длина n=27):
101001100111010111111011010
```

```
Введите кратность ошибки i (целое >0) [1]: 1

Внесены ошибки в позиции: [7]
Слово с ошибками Yn:
101001110111010111111011010

Синдромы:
  Строк:          [0 0 0 1 0 0 0 0]
  Столбцов:       [0 1]
  Общего:         0

Исправленные позиции: [np.int64(7)]
Исправленное слово Yn*:
101001100111010111111011010

>>> Ошибки успешно исправлены. Yn* совпадает с Xn.
```

5. Работа с 2 группами паритетов

```
Выбрано: k1=4, k2=4, k=16

Количество групп паритетов:
  2 — строки + столбцы
  3 — строки + столбцы + общий
  4 — строки + столбцы + главные диагонали + побочные диагонали
  5 — всё выше + общий паритет
Введите число групп (2-5) [3]: 2
Используется 2 групп паритетов

Введите информационное слово Xk (16 бит, только 0 и 1)
или нажмите Enter для случайного слова:

Сгенерировано: 0101101101011010

Информационная матрица Xk:
0 1 0 1
1 0 1 1
0 1 0 1
1 0 1 0

Кодовое слово Xn (длина n=24):
010110110101101001000001

Паритеты:
  Строковые:     [0 1 0 0]
  Столбцовые:    [0 0 0 1]
```

6. Количественный анализ

Для конфигурации 1:

```
Анализ корректирующей способности
Запустить анализ? (y/n): y
Число испытаний на каждую кратность [100]: 50
Максимальная кратность ошибки для анализа [3]: 2

Результаты (N1=50):
Кратность i  N2 (обнаружено)    N3 (исправлено)    N2/N1      N3/N1
1            50                 50                 1.000      1.000
2            50                 0                  1.000      0.000
```

Для конфигурации 2:

```
Анализ корректирующей способности
Запустить анализ? (y/n): y
Число испытаний на каждую кратность [100]: 50
Максимальная кратность ошибки для анализа [3]: 2

Результаты (N1=50):
Кратность i  N2 (обнаружено)    N3 (исправлено)    N2/N1      N3/N1
1            50                 50                 1.000      1.000
2            50                 0                  1.000      0.000
```

Код программы:

```
import numpy as np
```

```
import random
from typing import List, Tuple, Optional

def binary_string_to_matrix(binary_str: str, k1: int, k2: int) -> np.ndarray:
    if len(binary_str) != k1 * k2:
        raise ValueError(f"Длина строки должна быть равна {k1 * k2}")
    bits = [int(b) for b in binary_str]
    return np.array(bits).reshape(k1, k2)

def compute_row_parities(matrix: np.ndarray) -> np.ndarray:
    return np.sum(matrix, axis=1) % 2

def compute_col_parities(matrix: np.ndarray) -> np.ndarray:
    return np.sum(matrix, axis=0) % 2

def compute_main_diag_parities(matrix: np.ndarray) -> np.ndarray:
    """Паритеты по всем параллелям главной диагонали (i - j = const)."""
    k1, k2 = matrix.shape
    num_diags = k1 + k2 - 1
    diags = np.zeros(num_diags, dtype=int)
    for i in range(k1):
        for j in range(k2):
            d = i - j + (k2 - 1)          # сдвиг, чтобы индекс был >= 0
            diags[d] ^= matrix[i, j]
    return diags

def compute_anti_diag_parities(matrix: np.ndarray) -> np.ndarray:
    """Паритеты по всем параллелям побочной диагонали (i + j = const)."""
    k1, k2 = matrix.shape
    num_diags = k1 + k2 - 1
    diags = np.zeros(num_diags, dtype=int)
    for i in range(k1):
        for j in range(k2):
            d = i + j
            diags[d] ^= matrix[i, j]
    return diags

def compute_overall_parity(*parity_vectors) -> int:
    total = 0
    for vec in parity_vectors:
        total ^= np.sum(vec) % 2
    return total

def encode(info_matrix: np.ndarray, num_parity_groups: int = 3):
    """
    Кодирование.
    Возвращает:
      - codeword (плоский массив)
      - словарь со всеми паритетами (для отладки и декодирования)
    """
    k1, k2 = info_matrix.shape
    row_p = compute_row_parities(info_matrix)
    col_p = compute_col_parities(info_matrix)

    main_d = None
    anti_d = None
    overall = 0

    if num_parity_groups >= 4:
        main_d = compute_main_diag_parities(info_matrix)
        anti_d = compute_anti_diag_parities(info_matrix)

    if num_parity_groups == 3 or num_parity_groups == 5:
        vectors = [row_p, col_p]
        if main_d is not None:
            vectors.extend([main_d, anti_d])
        overall = compute_overall_parity(*vectors)
```

```
    # Формируем кодовое слово: info + row + col + [main] + [anti] + [overall]
    parts = [info_matrix.flatten(), row_p, col_p]
    if main_d is not None:
        parts.append(main_d)
        parts.append(anti_d)
    if num_parity_groups in (3, 5):
        parts.append(np.array([overall]))

    codeword = np.concatenate(parts)
    parities = {
        'row': row_p,
        'col': col_p,
        'main_diag': main_d,
        'anti_diag': anti_d,
        'overall': overall if num_parity_groups in (3, 5) else None
    }
    return codeword, parities

def introduce_errors(codeword: np.ndarray, num_errors: int) -> Tuple[np.ndarray,
List[int]]:
    n = len(codeword)
    num_errors = min(num_errors, n)
    positions = random.sample(range(n), num_errors) if num_errors > 0 else []
    yn = codeword.copy()
    for pos in positions:
        yn[pos] = 1 - yn[pos]
    return yn, sorted(positions)

def decode_and_correct(yn: np.ndarray, k1: int, k2: int, num_parity_groups: int = 3):
    """
    Декодирование и попытка исправления.
    Возвращает исправленное слово, список исправленных позиций и синдромы.
    """
    k = k1 * k2
    pos = 0

    # Разбираем принятое слово
    info_recv = yn[pos:pos + k].reshape(k1, k2)
    pos += k
    row_recv = yn[pos:pos + k1]
    pos += k1
    col_recv = yn[pos:pos + k2]
    pos += k2

    main_recv = anti_recv = None
    overall_recv = None

    if num_parity_groups >= 4:
        num_d = k1 + k2 - 1
        main_recv = yn[pos:pos + num_d]
        pos += num_d
        anti_recv = yn[pos:pos + num_d]
        pos += num_d

    if num_parity_groups in (3, 5):
        overall_recv = yn[pos]

    # Пересчитываем паритеты по принятым информационным битам
    row_calc = compute_row_parities(info_recv)
    col_calc = compute_col_parities(info_recv)
    main_calc = compute_main_diag_parities(info_recv) if num_parity_groups >= 4 else
None
    anti_calc = compute_anti_diag_parities(info_recv) if num_parity_groups >= 4 else
None

    # Синдромы
```

```
row_s = (row_recv + row_calc) % 2
col_s = (col_recv + col_calc) % 2
main_s = (main_recv + main_calc) % 2 if main_calc is not None else None
anti_s = (anti_recv + anti_calc) % 2 if anti_calc is not None else None

overall_s = 0
if num_parity_groups in (3, 5):
    vectors = [row_calc, col_calc]
    if main_calc is not None:
        vectors.extend([main_calc, anti_calc])
    overall_calc = compute_overall_parity(*vectors)
    overall_s = (overall_recv + overall_calc) % 2

# Поиск и исправление одиночной ошибки
yn_corr = yn.copy()
corrected_positions = []

error_rows = np.where(row_s == 1)[0]
error_cols = np.where(col_s == 1)[0]

# Случай 1: ошибка в информационном бите
if len(error_rows) == 1 and len(error_cols) == 1:
    r, c = error_rows[0], error_cols[0]
    # Дополнительная проверка по диагоналям (если они есть)
    consistent = True
    if main_s is not None:
        d_main = r - c + (k2 - 1)
        if main_s[d_main] != 1:
            consistent = False
    if anti_s is not None:
        d_anti = r + c
        if anti_s[d_anti] != 1:
            consistent = False

    if consistent:
        bit_pos = r * k2 + c
        yn_corr[bit_pos] = 1 - yn_corr[bit_pos]
        corrected_positions.append(bit_pos)

# Случай 2: ошибка только в строковом паритете
elif len(error_rows) == 1 and len(error_cols) == 0:
    r = error_rows[0]
    bit_pos = k + r
    yn_corr[bit_pos] = 1 - yn_corr[bit_pos]
    corrected_positions.append(bit_pos)

# Случай 3: ошибка только в столбцовом паритете
elif len(error_rows) == 0 and len(error_cols) == 1:
    c = error_cols[0]
    bit_pos = k + k1 + c
    yn_corr[bit_pos] = 1 - yn_corr[bit_pos]
    corrected_positions.append(bit_pos)

# Случай 4: ошибка в общем паритете
elif num_parity_groups in (3, 5) and overall_s == 1 and \
     len(error_rows) == 0 and len(error_cols) == 0 and \
     (main_s is None or np.all(main_s == 0)) and \
     (anti_s is None or np.all(anti_s == 0)):
    bit_pos = len(yn) - 1
    yn_corr[bit_pos] = 1 - yn_corr[bit_pos]
    corrected_positions.append(bit_pos)

syndromes = {
    'row': row_s,
    'col': col_s,
    'main_diag': main_s,
    'anti_diag': anti_s,
```

```
        'overall': overall_s if num_parity_groups in (3, 5) else None
    }
    return yn_corr, corrected_positions, syndromes

def analyze_correcting_ability(k1, k2, num_parity_groups, num_trials=50, max_errors=2):
    results = {}
    k = k1 * k2
    for i in range(1, max_errors + 1):
        N2 = N3 = 0
        for _ in range(num_trials):
            info_bits = [random.randint(0, 1) for _ in range(k)]
            info_str = ''.join(map(str, info_bits))
            info_mat = binary_string_to_matrix(info_str, k1, k2)
            xn, _ = encode(info_mat, num_parity_groups)
            yn, _ = introduce_errors(xn, i)
            yn_star, _, syndromes = decode_and_correct(yn, k1, k2, num_parity_groups)

            # Обнаружение: хотя бы один синдром ненулевой
            detected = (np.any(syndromes['row']) or np.any(syndromes['col']) or
                        (syndromes['main_diag'] is not None and
np.any(syndromes['main_diag'])) or
                        (syndromes['anti_diag'] is not None and
np.any(syndromes['anti_diag'])) or
                        (syndromes['overall'] is not None and syndromes['overall'] ==
1))
            if detected:
                N2 += 1
            if np.array_equal(yn_star, xn):
                N3 += 1
        results[i] = {
            'N1': num_trials,
            'N2': N2,
            'N3': N3,
            'N2_N1': N2 / num_trials,
            'N3_N1': N3 / num_trials
        }
    return results

def print_matrix(matrix, title):
    print(title)
    for row in matrix:
        print(' '.join(map(str, row)))

def main():
    print("Итеративное кодирование с поддержкой диагональных паритетов")
    print("Доступные конфигурации матрицы:")
    print("1. k1=4, k2=4 (2D)")
    print("2. k1=8, k2=2 (2D)")
    print("3. k1=4, k2=2 (упрощённый 3D-слой)")
    print("4. k1=2, k2=4 (упрощённый 3D-слой)")
    print()

    choice = input("Выберите конфигурацию [1]: ").strip() or '1'
    if choice == '2':
        k1, k2 = 8, 2
    elif choice == '3':
        k1, k2 = 4, 2
    elif choice == '4':
        k1, k2 = 2, 4
    else:
        k1, k2 = 4, 4

    k = k1 * k2
    print(f"\nВыбрано: k1={k1}, k2={k2}, k={k}")

    print("\nКоличество групп паритетов:")
    print("  2 — строки + столбцы")
```

```
print("  3 — строки + столбцы + общий")
print("  4 — строки + столбцы + главные диагонали + побочные диагонали")
print("  5 — всё выше + общий паритет")
num_str = input("Введите число групп (2–5) [3]: ").strip() or '3'
num_parity_groups = int(num_str)
if num_parity_groups not in (2, 3, 4, 5):
    num_parity_groups = 3
print(f"Используется {num_parity_groups} групп паритетов")

# Ввод информационного слова
print(f"\nВведите информационное слово Xk ({k} бит, только 0 и 1)")
print("или нажмите Enter для случайного слова:")
info_input = input().strip()
if not info_input:
    info_bits = [random.randint(0, 1) for _ in range(k)]
    info_str = ''.join(map(str, info_bits))
    print(f"Сгенерировано: {info_str}")
else:
    info_str = info_input
    if len(info_str) != k or any(c not in '01' for c in info_str):
        print("Ошибка ввода. Используется случайное слово.")
        info_bits = [random.randint(0, 1) for _ in range(k)]
        info_str = ''.join(map(str, info_bits))
        print(f"Сгенерировано: {info_str}")

info_matrix = binary_string_to_matrix(info_str, k1, k2)
print_matrix(info_matrix, "\nИнформационная матрица Xk:")

# Кодирование
xn, parities = encode(info_matrix, num_parity_groups)
print(f"\nКодовое слово Xn (длина n={len(xn)}):")
print(''.join(map(str, xn)))

print("\nПаритеты:")
print(f"  Строковые:     {parities['row']}")
print(f"  Столбцовые:    {parities['col']}")
if parities['main_diag'] is not None:
    print(f"  Главные диаг.: {parities['main_diag']}")
    print(f"  Побочные диаг.:{parities['anti_diag']}")
if parities['overall'] is not None:
    print(f"  Общий:         {parities['overall']}")

# Внесение ошибок
err_str = input("\nВведите кратность ошибки i (целое >0) [1]: ").strip() or '1'
num_errors = int(err_str)
yn, error_positions = introduce_errors(xn, num_errors)
print(f"\nВнесены ошибки в позиции: {error_positions}")
print(f"Слово с ошибками Yn:\n{''.join(map(str, yn))}")

# Декодирование
yn_star, corr_pos, syndromes = decode_and_correct(yn, k1, k2, num_parity_groups)

print("\nСиндромы:")
print(f"  Строк:         {syndromes['row']}")
print(f"  Столбцов:      {syndromes['col']}")
if syndromes['main_diag'] is not None:
    print(f"  Главных диаг.: {syndromes['main_diag']}")
    print(f"  Побочных диаг.:{syndromes['anti_diag']}")
if syndromes['overall'] is not None:
    print(f"  Общего:        {syndromes['overall']}")

print(f"\nИсправленные позиции: {corr_pos}")
print(f"Исправленное слово Yn*:\n{''.join(map(str, yn_star))}")

if np.array_equal(yn_star, xn):
    print("\n>>> Ошибки успешно исправлены. Yn* совпадает с Xn.")
else:
```

```
        print("\n>>> Полное исправление не удалось (характерно для кратности > 1).")

    # Анализ
    run = input("Запустить количественный анализ? (y/n) [y]: ").strip().lower() or 'y'
    if run != 'n':
        trials = input("Число испытаний на каждую кратность [50]: ").strip() or '50'
        max_err = input("Максимальная кратность ошибки [2]: ").strip() or '2'
        results = analyze_correcting_ability(k1, k2, num_parity_groups,
                                             int(trials), int(max_err))
        print(f"\nРезультаты (N1={trials}):")
        print(f"{'i':<4} {'N2 (обнар.)':<12} {'N3 (испр.)':<12} {'N2/N1':<8}
{'N3/N1':<8}")
        for i, res in results.items():
            print(f"{i:<4} {res['N2']:<12} {res['N3']:<12} "
                  f"{res['N2_N1']:<8.3f} {res['N3_N1']:<8.3f}")

    print("\nПрограмма завершена.")

if __name__ == "__main__":
    main()
```

Вывод: приобрела практические навыки кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.